RESEARCH STATEMENT

From Detection to Repair — Toward Long-Horizon, Trustworthy AI for Software Engineering

Jian Wang — Ph.D. Candidate, Nanyang Technological University

§1 Vision: Trustworthy Code Intelligence for the Age of AI-Assisted Programming

As large language models (LLMs) become the default authors of production code, the central problem of software engineering is undergoing a fundamental shift. The classical question—*"how do we help humans write correct code?"*—is increasingly displaced by a harder one: ***how do we verify, repair, and trust code written by machines whose internal logic we do not fully understand?*** My research builds the technical foundation for this new regime.

My work centers on **Trustworthy Code Intelligence (TCI)**—a research program integrating LLMs and retrieval-augmented methods into source code analysis to advance the *security, reliability, and sustainability* of modern software systems. The program is organized around three mutually reinforcing thrusts: (*i*) **AI-Generated Code Detection**—establishing trust at the source; (*ii*) **Automated Program Repair**—correcting faults at scale; and (*iii*) **Execution-Semantics-Grounded Reasoning**—moving beyond surface pattern matching toward genuine understanding of runtime behavior.

Looking forward, these three thrusts converge on a single frontier challenge: ***Long-Horizon Reasoning for software maintenance***—enabling AI to plan, iterate, and repair across multi-file, multi-step, dependency-heavy real-world scenarios *without frequent human intervention*. This is the bet I want to lead in the next five to ten years, and the structuring theme of my faculty research program.

§2 Past Contributions: A Three-Thrust Research Program

Over the past six years I have published in top-tier venues including ASE, ISSRE, EMNLP, ICML, NeurIPS, IJCAI, and TOSEM, with **four first-author papers** anchoring each of the three thrusts. The thrusts are not parallel projects; they are sequenced contributions that build toward the long-horizon vision in §3.

Thrust 1 — AI-Generated Code Detection: Trust at the Source

As LLM-produced code floods training corpora, codebases, and production systems, the first question is provenance: *can we tell what an AI wrote, and how reliable is that judgment?* My ASE 2024 paper (first author) is, to my knowledge, the first systematic study of AIGC detectors specifically on **source code content**. We curated multi-language benchmarks, evaluated state-of-the-art detectors, and exposed concrete failure modes—particularly under code transformation and across programming languages. The takeaway: ***detection without semantic grounding is brittle***, a finding that directly motivates Thrust 3.

This thrust is currently deployed in two real settings: (*a*) the Singapore **Trustworthy AI Centre (TAICeN)** national programme, where I lead the AIGC-Code Detection workstream; and (*b*) industry collaboration with

MetaTrust, where the technology supports code provenance verification and training-data quality control—filtering AI-generated contamination from enterprise codebases.

Thrust 2 — Automated Program Repair at Scale

Detecting that a piece of code is wrong is only the prelude; the harder problem is fixing it. My ASE 2025 paper, **Defects4C** (first author), introduces the first large-scale repair benchmark for **real-world C/C++ bugs**—a language family that dominates safety-critical systems (kernels, embedded software, scientific computing) yet was severely underrepresented in prior LLM-repair benchmarks. By stressing LLMs against memory errors, undefined behavior, and cross-file dependencies, Defects4C exposes the gap between benchmark accuracy on toy programs and capability on systems-level code.

Earlier in this thrust, my ISSRE 2024 paper **RATCHET** (first author) introduced a retrieval-augmented Transformer for program repair, integrating neural generation with symbolic retrieval to ground the model's output in real-code examples. Together, *RATCHET* and *Defects4C* form a methods-plus-evaluation pair that anchors my repair research, and they directly power the **CREATE** (Campus for Research Excellence and Technological Enterprise) national programme on medical-software security, where automated repair of clinical-system code is a primary safety requirement.

The unifying insight from this thrust: ***the bottleneck for repair is not generation but understanding.*** LLMs generate plausible patches readily; what they lack is a faithful model of how the buggy program actually behaves at runtime. This is precisely the question Thrust 3 attacks.

Thrust 3 — Execution-Semantics-Grounded Reasoning

Code is not just text—it is text with a behavior. My EMNLP 2025 (Findings) paper (first author) is the first comprehensive study of whether **execution traces**—the actual sequences of states a program visits at runtime—can be incorporated into code LLMs as an additional modality, and under what conditions doing so improves downstream reasoning tasks (program understanding, bug localization, repair). The empirical results are nuanced: traces help substantially for tasks with strong runtime structure, less so for purely syntactic tasks, and the gains depend on trace abstraction level. The methodological contribution is a systematic framework for trace-based augmentation that subsequent work can build on.

This thrust connects naturally to my earlier work on *Neuron Path Coverage* (TOSEM 2022, co-author), which introduced the idea of characterizing decision logic through execution paths inside neural networks. The intellectual through-line—*treat the trace as the missing modality*—connects neural-network testing and program understanding into a single research direction.

§3 Future Agenda: Three Horizons toward Long-Horizon Software AI

My past work establishes the methodological building blocks; my future agenda assembles them into the larger goal of long-horizon, trustworthy AI for software engineering. I plan three nested horizons over the next decade.

Horizon A (1–3 years) — Verifiable AI-Assisted Programming

Integrate Thrusts 1 and 2 into a closed-loop, *verification-grounded* AI-assisted programming pipeline: detect when code is AI-generated → verify against lightweight specifications (property-based testing, symbolic execution snippets) → repair when verification fails → re-verify. The novelty is not any single component but the loop, and especially the use of execution-trace evidence from Thrust 3 to guide both verification and repair. Concrete first deliverable: an open benchmark and tool ecosystem extending Defects4C to **repository-level multi-file repair**, with verification artifacts as first-class evaluation criteria.

Target funding mechanisms include the NSFC General/Young Scholars program, the MOE Tier-2, and Singapore NRF/CSA cybersecurity calls—the latter two of which my advisor and I have recently won and which I can extend in a new lab.

Horizon B (3–5 years) — Execution-Grounded Code Foundation Models

Rethink the training paradigm of code LLMs from *text-only* to ***text + trace + program state***. Today's strongest code LLMs are pretrained almost exclusively on static source text; runtime behavior is, at best, an afterthought injected via post-hoc tool use. Horizon B explores integrating execution evidence at *pretraining* time—designing tokenization schemes for trace data, multimodal attention architectures bridging static and dynamic views, and curriculum strategies that move models from short to long horizons. This bridges program languages and ML systems, opening collaborations with the ML community while remaining grounded in software-engineering problems.

Horizon C (5–10 years) — Long-Horizon AI for Safety-Critical and Scientific Software

The synthesis. I aim to develop AI systems that can autonomously sustain **long-horizon software maintenance**: following a high-level intent (e.g., “migrate this codebase to a new ABI” or “fix all uses of a deprecated cryptographic primitive across this kernel module”), and carrying out the multi-step plan—locating affected sites, repairing each, verifying global invariants, regenerating tests—with minimal human intervention. The natural application domains are precisely those that suffer most from manual maintenance cost: *autonomous driving stacks, medical devices, scientific computing, and critical-infrastructure firmware*. This horizon is also where the social impact of the program is most visible, and where I expect to attract industrial partnerships and PhD students motivated by real-world consequences.

§4 Why This Program, Why Now

Three converging trends make this program timely. First, *AI-generated code is no longer an experimental phenomenon*—industry estimates place it at a non-trivial fraction of production commits, and the trust gap is widening faster than current detection or repair tooling can close. Second, *long-horizon reasoning has emerged as a defining frontier* in both ML and SE, but most published work is still on stylized benchmarks rather than real codebases—there is a clear opening for SE researchers to set the agenda. Third, governments and industry are now funding *trustworthy-AI infrastructure* at unprecedented scale (Singapore's TAICeN,

CyberSG, and IAF-ICP programs are concrete examples I have already participated in)—there will be both the resources and the policy interest to support an ambitious program in this space.

My background is unusually well-matched to this opportunity. I bring **six years of industrial systems experience** (Xiaomi AI Lab, 58.com) before entering academia, including delivering algorithms to production on millions of devices and leading teams at 15M-MAU scale. This gives me firsthand intuition for the failure modes of real software at scale—intuitions that complement, rather than replace, the methodological training I received during my PhD at NTU. I have demonstrated that I can take a research idea from concept (RATCHET, Defects4C) through national-program deployment (TAICeN, CREATE) and industrial uptake (MetaTrust), and I have co-founded a startup (VAISION) capitalized through competitive challenge prizes. The same end-to-end posture is what I would bring to the lab I build.

§5 Teaching and Mentoring Plans

I plan to develop and teach core undergraduate courses in software engineering, programming languages, and compilers, and a graduate seminar I am particularly motivated to design: ***LLMs for Software Engineering: Foundations, Evaluation, and Trust.*** The seminar would pair classical SE foundations (specification, testing, repair) with hands-on assessment of modern code LLMs, equipping students to be *informed builders and rigorous evaluators* of AI-assisted development tools—a profile industry urgently needs and academia has not yet supplied at scale.

I will mentor students with the same end-to-end posture I aspire to in my own research: every PhD project should pass through three checkpoints—a publishable methodological contribution, a deployable artifact, and an attempt at real-world uptake (industry, government, or open-source community). This is the model that produced my own strongest results, and I believe it is reproducible.

§6 Closing

Software is the infrastructure of modern life, and AI is increasingly the author of that infrastructure. The research program I propose—trustworthy code intelligence advancing toward long-horizon reasoning—addresses what I believe will be one of the defining intellectual challenges of the coming decade, while also producing the kind of graduates and artifacts that industry, government, and society actively need. I would be delighted to discuss how this program could be developed at your institution, and how it might align with existing strengths in software engineering, machine learning, and trustworthy systems.